

货币的本质：信用(金本位、国家信用、布雷顿森林体系)

货币理论：哈耶克经济学，货币非国有化。现代货币理论MMT，货币的本质就是欠条。

哈希算法：快速定位，错误校验，唯一性验证。逆向困难，抗碰撞。

SHA256：对于任意长度的输入数据，SHA256都会产生一个256bit长的哈希值，通常用一个长度为64的十六进制字符串来表示。

第一部分、区块链技术原理

比特币一种点对点的电子现金系统。

数字货币面临的问题：虚拟货币(数字签名解决)，多重支付(分布式账本解决)。

分布式账本两个核心技术：以链式区块组织账本数据实现账本数据的不可篡改，分布式的可信记账机制。

区块链宏观结构：基于区块链进行连接。区块链中数据无法篡改或删除，区块链越长可信度越高。

区块链微观结构：每个区块包括区块头和交易数据两个部分。区块头由当前区块的元数据和前一个区块的Hash值构成，Merkle树用于对交易数据列表进行快速寻址。

区块标识符：区块头哈希值(通过SHA256算法进行二次哈希计算得到的数字指纹)和区块高度(非唯一，可能出现分叉)。

Merkle树：叶节点是数据块的哈希值，非叶节点的哈希值是根据它下面子节点的哈希值计算得到。

Merkle树被用来归纳一个区块中的所有交易，同时生成整个交易集合的数字指纹，且提供了一种校验区块是否存在某交易的高效途径。当N个数据元素经过加密后插入Merkle树时，你至少需要log2(N)次就能检查出任意某数据元素是否在该树中。

Merkle树的价值：快速比较大量数据，快速定位修改，快速验证其中数据。

比特币区块链中的节点：全节点(包含矿工和非矿工节点)，存储着整个区块链，承担对交易请求进行验证和执行，可以通过挖矿争取发布区块，还承担着应别的节点之请向其发送区块和相关交易信息的义务，同时也承担转发交易请求和区块的义务。轻节点，简单支付验证(SPV)节点只存储着整个区块链，承担对交易请求进行验证和执行，可以通过挖矿争取发布区块，还承担着应别的节点之请向其发送区块和相关交易信息的义务，同时也承担转发交易请求和区块的义务；钱包记录所有者有关的信息：区块链地址、私钥、账户余额、UTXO等，不存储账本。

SPV 验证过程：

- 针对某个支付到自己比特币地址的交易建立布隆过滤器，限制只接收含有目标比特币地址的交易。

- 其他全节点探测到某个交易符合SPV节点设置的布隆过滤器条件时，以Merkleblock消息的形式发送该区块，Merkleblock消息包含区块头和一条连接目标交易与Merkle根的路径。

- 交易的存在性验证：SPV节点通过该Merkle路径找到跟该交易相关的区块，并验证对应区块中是否存在目标交易(Merkle Path Proof)。

- 交易是否双花验证：SPV节点检查这笔交易所在区块之后的区块个数，区块个数越多说明该区块被全网更多节点共识，一般来说，一笔交易所属区块之后的区块个数达到6个时，说明这笔交易是可信的。

共识机制：

- 目的：解决记账权
- 场景：开放系统，动态增减，海量节点
- 解决方案：工作量证明，高强度哈希计算(SHA256)进行算力竞争解决记账权，达成共识。

拜占庭将军问题：

1. 每个分布式节点独立运行
 2. 相互间可通过网络通信
 3. 可能存在恶意节点
 4. 信道是可靠的，但可能中断
- 目标：一致性和正确性。

比特币区块链共识协议模型：

- 划定固定时间段(10分钟)
- 相同或相似输入数据(组装的区块)
- 算力竞争选出获胜节点，其它节点验证结果后不再发送消息
- 最长链原则，从短期共识扩展到长期共识
- 工作量证明的难度设定和调整：New Difficulty = Old Difficulty * (Actual Time of Last 2016 Blocks / 20160 minutes)

区块生成和组装：

- 竞争胜出的节点创建新区块并广播
- 其他独立节点检验新区块
- 区块链的组装和选择(根据区块hash值查找父区块)
 - 连接到主链上
 - 分叉(备用链)
 - 孤立区块

区块链分叉：每个节点独立地对新区块进行校验并组装进区块链。每个节点对区块链进行独立选择，在工作量证明机制下选择累计工作量最大的区块链。

挖矿：

- 挖矿节点必须有钱包功能
 - 有自己的160位密码地址、私钥
- 打包成区块时，区块中额外加一个交易coinbase
 - 生成一个UTXO，包含当前奖励数量的比特币
 - 这个UTXO的领袖地址是自己的地址
 - 如果记账成功，这个coinbase交易就生效，否则不在链上

UTXO(未花费的交易输出)模型：每个地址的资金余额就是散布在账本中所有UTXO的总和，使用时把自己名下的UTXO作为交易输入，可能需要拼凑花费。

比特币中有两类交易：常规交易，有交易输入(支付者地址和金额)、交易输出(收入者地址和金额)，挖矿交易(Coinbase)，产生比特币，只有交易输出(挖矿者地址和金额)。

比特币虚拟机：运行认领脚本或者认领脚本。指令集能力有限，主要验证签名是否正确。

认领脚本：P2PK(Pay to Public Key), P2PKH(Pay to Public Key Hash), P2SH(Pay to Script Hash), P2WPKH(Pay to SegWitness Public Key Hash), P2WSH(Pay to SegWitness Script Hash)

区块链技术是一种以非对称加密技术对交易进行数字签名，通过工作量证明等共识机制进行记账节点协调，数据以链式区块形式组织存储的分布式账本技术。

区块链技术的优点：

- 去中心化：避免垄断，点对点交易，去代理
- 数据公开：无暗箱操作，平等性，开放生态体系
- 可信：数据永久可靠，记录可信

区块链的问题：

- 隐私问题
 - 匿名地址如何监管
- 性能问题
 - 交易确认时间长
 - 区块容量有限
- 系统性风险
 - 区块链分叉和51%攻击

支付通道：用于链外交易或其他交易，面向性能要求高、交易数量多等场景。包含注资交易(Funding Transaction)、应交易(Commitment Transaction)、联合交易(Settlement Transaction)、退款交易(Refund Transaction)。

闪电网络：还包含转存交易(Delivery Transaction)，任何一方都可以从联合地址认领属于自己的资金，把它转到一个自己更方便花费的地址中。可撤销转存交易(RevocableDeliveryTransaction)，可以撤销。补救交易(BreachRemedy)，这是在对对方违约情况下加以补救并使对方受惩罚的交易。

第二部分、数字货币和区块链生态

区块链形态：联盟链，对产业或者国家的特定清算和结算用途，容易进行控制权限定，更高的可扩展性。公链，任何人都可以参与，容易部署应用程序，全球范围可以访问，不依赖于单个公司。私链：由单链的个体或者组织拥有，对组织内部的审计和测试有用。也可分为无许可区块链和许可区块链。

ICO(Initial Coin Offering)意为首次代币发行，本质是一种产品众筹，是项目对构建基于区块链的产品或服务的承诺。区块链项目完成之前，通过公开售卖部分平台的Token募集资金用于创始团队的开发工作。

通证(Token)：一个区块链生态里的一种权益证明。在经济体系下常被称为区块链生态内的流通的货币，如比特币。以太坊，通常情况下，Token一经发行，便严格按照区块链代码执行，不受个人或机构控制。

智能合约：当满足一定条件，区块中的程序逻辑会被触发执行合同约定的指令(比如资产清算、赔偿、交割等)。

数字货币钱包：“钱包”是指用于存储和管理用户私密容器的，钱包里没有“钱”，钱包软件一般还控制用户访问权限，管理密钥和地址，跟踪余额以及创建和签署交易。

第三部分、以太坊技术分析

以太坊的组成：P2P网络，交易，状态机，区块链账本，共识算法，客户端。

以太坊状态：

- 节点：以太坊是由网络上的许多节点组成的，每一个节点都运行着以太坊虚拟机，通过节点可以访问以太坊数据的读写。节点之间使用共识机制来确保数据交易的可靠性和正确性。

- 账户：以太坊中包含两类账户，包括 外部账户和 合约账户。外部账户由公私钥对控制，合约账户则在区块链上唯一的标识了某个智能合约。

- 交易：交易包括转账交易和合约交易，通过状态转移来标记。状态由账户之间的转移价值和信息状态转移构成。

- 智能合约：合约是代码和数据的集合，存在于以太坊区块链的指定地址。合约方法支持回调操作，如果在执行某个方法时发生异常(如gas消耗完)，则该方法已经执行的操作都会被回滚。但是如果交易一旦执行完毕，是没有办法篡改的。

- EVM：以太坊虚拟机是以以太坊中智能合约的运行环境，并且是一个沙盒，与外界隔离。智能合约代码只在 EVM 内部运行时，是不能进行网络操作、文件 I/O 或者执行其它进程的。智能合约之间也只能进行有限的调用。

- 矿工与挖矿：矿工是指通过不断重复哈希运算来产生工作量的网络节点。在以太坊中，发行以比特币的唯一途径是挖矿。

- gas：以太坊上的每一笔交易都有矿工的参与，且都需要支付一定的费用，这个费用在以太坊中称为 gas。gas 的目的是限制执行交易所需的工作量，同时为执行交易支付费用。

外部账户和合约账户的区别：

- 外部账户：
 - 创建外部账户没有成本
 - 可以启动交易
 - 两个外部账户间的交易只能是以比特币转账
 - 由公私钥对控制
- 合约账户：
 - 创建合约账户需要成本，因为使用了网络存储
 - 只能在收到一个交易作响应而发出一个交易
 - 从一个外部账户到一个合约账户的交易可以触发合约账户上的代码，执行代码中的各种动作，例如代币转账、创建新合约等
 - 合约帐户没有私钥，相反，它们由智能合约代码的逻辑控制

账户间的交易：

- 外部账户可以向其他外部账户发送消息，或向其他合约账户发起一个合约交易。
- 两个外部账户间的消息就是一个简单的转账交易。
- 从一个外部账户到一个合约账户的消息可以触发 合约账户上的合约代码执行。
- 合约地址自身不能发起一个新的交易，只能在接收到其他外部账户或合约账户发起的交易后作出反应而产生内部交易。

世界状态：所有账户的状态合集。转换状态的最小单元是交易(原子性、一致性)，每次都执行一条或多条交易后发生状态转换。

世界状态是由账户组成的，每一个账户都有一个唯一的地址。地址规则：账户公钥计算所得/部署者的信息计算所得。

交易类型：

- 简单支付交易：以太币的账户间转账。
- 验证交易：在简单支付交易中把支付额设置成 0 需要存证的内容写在 data 字段中就成了存证交易。
- 合约部署交易：将对方地址设置成 0 就可以将智能合约的程序(data 字段)部署到以太坊网络中 实际上是所有的验证节点上。
- 合约调用交易：对智能合约的调用(通过 data 字段)，(包括复杂支付交易(Token 的转移)，查询交易，其他交易(电子商务、电子政务等))。

Gas 是用来计量以太坊系统资源使用情况的的最小计量单位，超过该系统中所有动作都有一个复杂度衡量的公式，一次交易调用过程，累计消耗 Gas 状态转移者预付的总量交易将失败。

Fee = Gas * GasPrice 交易产生的手续费作为 Block Producer 的经济激励。

因为以太坊状态数据库在每个节点均有存储，为鼓励较少的存储数据量，如果一个交易清除存储中的一项，这项操作的执行费用被免除，并有奖励费用。

base fee：最小 gas 单位。priority fee：支付的小费 gas 单位。max fee：最大 gas 单价。

区块大小可变，每个区块的目标大小是容纳至少消费 15M gas 的交易，但区块的大小将根据网络需求增加或减少，上限是消费 30M gas 的交易(目标区块大小的 2 倍)。Hash 函数会产生较多的碰撞，产生较多的区块链分叉，所以会挑选计算量最大的那条路径。

回找：区块编号，区块哈希，交易哈希，当前交易消耗的 gas，当前交易执行后当前区块中使用的累计 gas，执行当前交易创建日志

日志：合约编码器可以在智能合约中定义 Event，当智能合约运行中执行该语句就会产生一个虚拟机日志，并储存在回找中。

以太坊使用 Merkle Patricia Trie 储存数据，(1) 将根节点传入作为当前处理节点 传入目标节点的 Key 作为路径path。(2) 传入新的 Value 值 若 Value 值为空 则找到该节点并删除 反之创建一个新节点替换旧节点。

以太坊虚拟机 EVM 是一个基于栈的解释器，用于解释执行合约账户字节码。EE 是 EVM 的执行环境，包括环境参数和存储空间读写函数。拥有完全隔离的执行环境，无法访问宿主机的系统资源。

EVM 有一个固定的指令集，包括算数运算、比特运算、逻辑运算、跳转指令、状态读取、存储指令。

图灵完备是针对一套数据操作规则而言的概念。数据操作规则可以是一门编程语言，也可以是计算机机里具体实现的指令集。当这套规则可以实现图灵机模型里的全部功能时，就称它具有图灵完备。

EVM 包含三大模块：

- 编译合约模块：主要是对底层 Solc 编译器进行一层封装 提供 RPC 接口给外部服务，调用 Solidity 编写的智能合约进行编译。
- Ledger 模块：主要是对区块链账户系统进行修改和更新 账户一共分为两种，分别是普通账户和智能合约的账户 调用方如果知道合约账户地址则可以调用该合约账户的每一次修改都会被持久化到区块链中。

- EVM 执行模块：主要功能是对交易中的智能合约代码进行解析和执行，一般分为创建合约调用和合约两部分。EVM 执行模块除了支持普通的字节码执行外还支持 JIT 模式的指令执行(优化后的指令)。

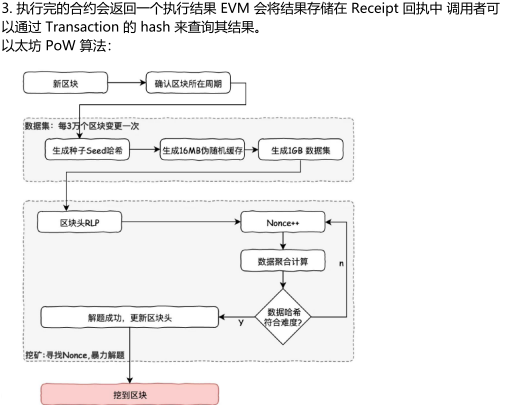
EVM 执行流程：

1. EVM 接收到 Transaction 信息 然后判断 Transaction 类型是部署合约还是执行合约 如果是部署合约 则新建一个账户来存储合约地址和编译后的代码；如果是执行合约或是调用合约 则使用 EVM 来执行输入指令集。

2. 执行上一条指令集之后判断 EVM 是否停机 如果停机则判断是否正常停机 正常停机则更新合约状态到区块链 否则回滚合约状态。 如果不停机则继续执行下一条指令集 重复 2。

3. 执行完的合约会返回一个执行结果 EVM 会将结果存储在 Receipt 回找中 调用者可以通过 Transaction 的 hash 来查询其结果。

以太坊 PoW 算法：



以太坊 PoS 算法：

- 验证者以 ETH 的形式将资本抵押给以太坊上的一个智能合约。抵押的 ETH 充当抵押品，如果验证者行为不诚实或懒惰，可被销毁。然后，验证者负责检查通过网络传播的新块是否有有效，并偶尔创建和传播新块。

- 要成为验证者，用户必须将 32 ETH 存入存款合约并运行个软件：执行客户端，共识客户端和验证器。在存入 ETH 时，用户加入一个激活队列。

- 一旦激活，验证者就会从以太坊网络上的对等方接收新的区块。重新执行区块中交付的交易，并检查区块签名以确保区块有效。然后验证者在网络上发送支持该块的投票(称为证明 attestation)。

- 以太坊信标链 Beacon Chain：

- 每 12s 为一个时间 slot
- 每 32 slot 为一个纪元 epoch
- 每个 slot 产生一个新区块，但可能没有
- 信标链的 Genesis 区块在 slot 0
- 每个时段 slot 有一组验证者组成委员会
- 其中一些验证者被随机选中成为区块的 proposer，发出区块
- 委员会的其他成员负责 attestation，如 2/3 支持 则该区块发布
- 每个 validator 在每个 epoch 只能属于一个委员会。通常有超过 8192 个验证者，这意味着每个 slot 都有一个以上的委员会。所有委员会的规模相同，至少有 128 名成员。当验证者少于 4096 个时，安全性会降低，委员会成员将少于 128 个。
- 每个 epoch 验证者被平均分配到各个 slot 中然后再细分为适当规模的委员会。洗牌算法会增加或减少每个 slot 的委员会数量，以使每个委员会至少有 128 个成员。

检查点：

- 检查点是 epoch 的第一个 slot 中的第一个区块，如果没有这样的区块，则检查点是前面最近的区块。一个区块可以是多个 epoch 的检查点。
- 当验证者提交 LMD GHOST 投票时，他投票当前 epoch 的 checkpoint，称为 target。

- 这个投票称为 Casper FFG 投票，还需要包含前一个 checkpoint，称为 source。一个 epoch 结束时，如果它的 checkpoint 获 2/3 验证者投票支持，其状态改为 justified

- 一个 justified 的 checkpoint 如果其后续的 checkpoint 状态改为 justified，则它的状态升级为 finalized

- 一般情况下一个 checkpoint 经过 2 个 epoch 后(12.8 分钟)变为 finalized

投票接收协议：生成 > 传播 > 聚合 > 传播 > 接纳

第四部分、DApp 是什么

DApp 架构：

- 前端：数据显示
- 区块链和智能合约：处理和存储用户关心的核心数据，并让数据和用户绑定，在 App 之间共享
- 传统后端数据库：处理和存储复杂和不那么重要的数据，主要用于优化应用

EVM与智能合约

智能合约：每个人都能看到且无法篡改，云函数可以处理业务逻辑并存储数据

部署合约：

- 用户部署合约
- 编译合约，生成字节码文件，发送部署合约的交易到节点
- 节点验证合法性，创建合约账户并部署合约
- 节点广播交易，争夺记账权，交易被确认
- 执行/调用合约：
- 用户建立交易体，对交易签名
- 用户将数据发送到节点
- 节点验证、运行合约中的相应方法，并返回执行交易的回找
- 节点广播交易，争夺记账权，交易被确认

EVM 基本要求：

- 确定性：
 - 所有节点需要运行相同指令得到相同结果
 - 无非确定性的系统调用
 - 无浮点数精度
- 不可终止性
 - 停机问题
 - 区块链上的交易执行是串行的，若某计算问题算法无法在有限步内完成，将阻塞整个区块链系统
- EVM 是“准”图灵完备(图灵完备包含死循环)，通过手续费将程序限制在有限的计算步骤内
- 隔离性
 - 保证智能合约的执行环境是受限的

如果程序执行到一半 Gas 消耗完了怎么办？

交易执行具有原子性：交易终止，所有状态修改全部回滚不退还

如果程序执行完了 Gas 还没消耗完怎么办？

交易费退还

Gas Limit 是否可设为无限大？

不行，以太坊对每个区块有 gas limit 的限制

EVM 中各个组件：

- PC：类似于 CPU 寄存器
- Stack：堆栈，位宽 256bits
- Memory：内存空间，暂时存储
- Storage：永久的存储空间
- Gas：油费池

存在问题：

- 安全问题
- 难以支持复杂杂度的运算(例如密码学运算、数学运算、浮点运算、随机算法、http 请求)

Precompiled 合约

- 将复杂函数封装成系统调用

- 当 EVM 检测到 Precompiled 合约被调用时，跳出 EVM，用宿主主机来调用高度优化的系统函数进行执行

第五部分、超级账本技术分析

社区组织结构：技术委员会，负责技术相关的工作，下设多个工作组，具体带动各个项目和方向的发展。管理董事会，负责社区组织的整体决策，由超级账本会员中推选出代表。Linux 基金会，负责基金管理，协助 Hyperledger 社区在 Linux 基金会的支支持下发展。

Fabric 是一个面向企业应用的区块链软件栈，不是公链，不发行加密货币。

Fabric 的主要设计特性：

- 资产：资产定义允许通过网络交换几乎任何具有货币价值的东西。
- 链码：链码执行与事务排序分开，限制节点类型之间所需的信任 and 验证级别，并优化网络可扩展性和性能。
- 账本：不可变的共享账本对每个通道的整个交易历史记录进行编码，并包括类似 SQL 的查询功能，可实现高效的审计和争议解决。
- 隐私：通道和私人数据收集支持私人和多边的交易，这些交易通常是在公共网络上交换资产的竞争企业和受监管企业所需要的。
- 安全 & 会员服务：许可会员资格提供了一个可信的区块链网络，参与者知道所有交易都可以被授权的监管机构和审计师检测和跟踪。

- 共识：一种独特的共识方法可实现 企业 所需的灵活性和可扩展性。

Fabric 架构：

- 成员服务：会员注册、身份保护、内容保护、交易审计功能。
- 区块链服务：节点共识管理、账本分布式计算、账本存储、节点间 p2p 协议功能实现。

- 合约代码服务：提供智能合约的服务。

成员服务：

- 支持一个交易网络，在这个网络中，所有参与者都拥有已知的身份。
- 公钥基础设施(PKI)用于生成与组织、网络组件以及终端用户或客户端应用程序相关联的加密证书。可以在更广泛的网络和通道级别上提取和管理数据访问控制。

- 成员服务提供者(MSP)分发证书、验证证书和用户授权背后的所有加密机制和协议抽象出来。MSP 可以定义它们身份概念，同样还可以定义管理身份验证和认证签名生成和验证这些身份的规则。

CA(Certificate Authority)体系：PKI 由各方(如服务的用户、服务提供者)发布数字证书的数字授权中心组成，各方使用证书在其与服务交换的消息中对自己进行身份验证。Fabric 有一个独立的 Fabric CA 模块，用来管理证书服务，专门进行证书发放、管理和身份验证，当然 Fabric 也允许第三方 CA 机构的接入。

CA 证书:

- 一个自签名 CA 证书列表来组成信任根
- 一个 X.509 证书列表来代表证书验证时需要考虑的中间证书, 这些证书应该由某一个信任根颁发; 中间证书是可选的参数
- 一个 X.509 证书列表, 并拥有从某一信任根起可验证的 CA 证书路径, 来代表该 MSP 的管理员证书; 拥有管理员证书则代表拥有申请改变该 MSP 配置的权力
- 一个组织单位列表, 此列表应出现在该 MSP 的有效成员的 X.509 证书中; 这是一个可选的配置参数
- 一个证书撤销列表 (CRLs), 其中每一个对应一个列出的(根或中间)MSP CA ; 这是一个可选参数
- 一个自签证书列表, 用来组成 TLS 证书的信任根 TLS root of trust
- 一个 X.509 证书列表来代表证书验证时需要考虑的 TLS 中间证书, 这些证书应该由某一个 TLS 信任根颁发; TLS 中间证书是可选的参数

Fabric CA 提供客户端和 SDK 两种方式与 CA 进行交互, 每个 Fabric CA 都有一个根 CA 或者中间 CA 。为了保证 CA 的安全性, 中间 CA 可以采用集群方式搭建。

区块链服务:

- 1.P2P 协议: Peer 和 Orderer 采用 gRPC 提供远程服务给客户端调用, 节点间使用 Gossip 协议进行状态同步和分发。
2. 共识机制: 支持 SOLO、Kafka、Raft 三种。
3. 分布式账本: 包括世界状态和交易日志。
4. 账本存储: 支持 LevelDB、CouchDB。

Fabric 的节点:

- Client: 至少连接一个 peer 或者 orderer 节点, 一般只保存与自己有关的账户数据
- Orderer: 编排节点, , 接收包含背书签名的交易, 对未打包的交易进行排序生成区块, 并广播给 Peer 节点
- Peer: 对等节点, 通过执行链码对账本进行读写, 都是 Committer, 维护状态数据和账本副本
- Endorser: 背书节点, 执行交易并对结果进行签名背书, 是动态的, 需要链码指定哪些节点点对交易背书
- Committer: 提交节点, 接受生成的区块, 验证后会以附加的方式提交到账本副本中

智能合约的背书策略明确了在交易被接受并且记录到提交节点的账本之前, 需要哪些组织的 Peer 节点为交易签名。

区块链网络: 多个组织聚集在一起形成一个通道, 事务在链码中调用, 并且权限由最初配置通道时商定的策略确定

Channel: 通道是构建在 Fabric 区块链网络上的由若干节点所组成的一条链, 实现了数据的私有和保密。

一个 Peer 节点至少接入一个通道, 可以接入多个通道。每个通道拥有自己单独的账本, 且仅对通道成员节点共享。

Orderer 节点拥有的通道成为系统通道, 该通道账本拥有网络上所有账本数据。节点账本可以被创建, 并绑定到一个通道。

主节点: 有一个主节点, 它负责将交易从排序节点分发到该组织中其他的提交节点锚节点: 如果一个 Peer 节点需要同一个组织的 Peer 节点通信的话, 它可以使用对方组织通道配置中定义的锚节点。一个组织可以拥有 0 个或者多个锚节点, 并且一个锚节点能够帮助很多不同的跨组织间的通信。

世界状态: 是一个数据库, 它存储了一组账本状态的当前值。默认情况下, 账本状态是以 键值 对的方式来表示的。因为我们可以创建、更新和删除状态。

区块头: 包括区块编号、哈希值、前一个区块哈希值

区块数据: 这部分包含了一个有序的交易列表。区块数据是在排序服务创建区块时被写入的。

区块元数据: 这个部分包含了区块被写入的时间, 还有区块写入者的证书、公钥以及签名。随后, 区块的提交者也会为每一笔交易添加一个有效或无效的标记, 但由于这一信息与区块同时产生, 所以它不会被包含在哈希中。

交易结构: 头、签名、提案、响应 (以读写集的形式记录下世界状态之前和之后的值)、背书 (一组交易签名响应, 这些签名都来自背书策略规定的相关组织, 并且这些组织的数量必须满足背书策略的要求)

链码: 使用 Docker 容器来运行, 是图灵完备的, 设定定时器防止死循环

背书: 它指明了区块链网络中哪些组织必须对一个给定的智能合约所生成的交易进行签名, 以此来宣布该交易有效。

交易验证:

智能合约提取一组名为交易提案的输入参数, 并将其与程序逻辑结合起来使用以读写账本。对世界状态的更改被捕获为交易提案响应, 该响应应包含一个读写集, 其中既含有已读取的状态, 也含有还未书写的最新状态 (如果交易有效的话)。

注意, 在执行智能合约时世界状态没有更新

验证步骤: 1.根据背书策略检查交易。2.确保交易在受到背书节点签名时他的交易到背书节点签名时它的交易读集与世界状态的当前值匹配, 并且中间过程中没有被更新。

注意, 所有交易 (有效/无效) 都会被添加到区块链历史中

智能合约主要进行读取 (查询对象)、写入 (修改对象)、删除 (移除对象)

链码部署:

1. 打包链码, 由通道内一个组织或多个组织完成
2. 在 peer 节点上安装链码, 由 peer administrator 完成, 有问题则返回错误
3. 批准组织的链码定义, admin 发起, 通过后提交给 ordering 服务, 分发给所有 peer 批注后将存储
4. 提交链码定义到通道, admin 发起, peer 背书后提交给 ordering 服务, 然后提交给通道

Fabric 共识组成 (确定性的, 不会有分叉):

- 提案与背书

用 fabric gateway 服务把提案发送给一个 peer 节点, 该节点执行提案或者交给别人执行, 然后发给一组节点背书。根据策略, 背书节点进行签名然后返回一个提案响应。

- 排序

到达排序节点的每个授权交易都被机械地打包在一个区块中, 排序节点不判断交易的内容。

- 验证与提交

Peer 节点可以使用 gossip 协议 将区块 关联到其他节点。

排序服务:

- Solo 是一种部署在单个节点上的简单时序服务 它只支持单链和单通道
- Kafka 是一种支持多通道分区的集群共识服务, 可以支持 CFT Crash Faults Tolerance, 它容忍部分节点宕机失效, 但是不能容忍恶意节点
- Raft 是一个分布式崩溃故障容错, 管理复制日志的共识算法 它可以保证在系统中部分节点出现非拜占庭故障的情况下, 系统依然可以处理客户端的请求

容错共识对比:

- 拜占庭容错 (崩溃+恶意): 节点总数 3n+1 时, 出错节点不超过 n (PBFT/Hotstuff)
- 崩溃容错: 节点总数 2n+1 时, 出错节点不超过 n (Raft/Kafka)

复制状态机工作流程

1. 客户端向主导节点Leader Node发送包含命令的请求。
2. 主导节点将收到的请求追加到其日志中, 并将该请求发送给所有的跟随节点Follower Node。跟随节点也会将该请求追加到自身的日志中并返回一个确认消息。
3. 一旦主导节点收到大部分跟随节点的确认消息, 就会将命令日志提交给其管理的状态机。一旦主导节点提交了日志, 跟随节点也会将日志提交给自身管理的状态机
4. 主导节点向客户端返回响应结果

节点的状态:

- 跟随状态: 初始状态, 一旦某个跟随节点没有正常通信, 就会转换成候选状态。
- 候选状态: 会发起选举, 如果收到大多数节点的投票, 就会成为主导节点。
- 主导状态: 处理客户端请求并确保所有跟随节点日志副本相同。
- 任期: 用来标识主导节点的管理周期, 每个任期都从选举开始直到下一个任期
- 如果主导节点或候选节点发现另一主导节点任期更高会退回到跟随状态

Raft 使用心跳机制来触发主导节点的选举。当节点启动后进入跟随状态, 只要它能从主导节点或候选节点收到有效的 RPC 心跳消息, 就会保持在跟随状态。主导节点会周期性发送心跳消息给所有的跟随节点来维持其主导地位。如果某个跟随节点在一段时间内没有收到心跳消息, 就发生选举超时事件, 该节点就认为目前没有主导节点并发起选举来选出新的主导节点。

排序节点工作流程:

1. 交易自动路由到通道的当前主导 orderer 节点
2. 主导 orderer 节点 验证交易后, 将收到的交易传入区块切割模块, 创建候选区块
3. 产生新的区块, 主导 orderer 节点 将其应用于本地的 Raft 有限状态机
4. 有限状态机将尝试复制到足够数量的 orderer 节点, 以便提交区块
5. 区块被写入接收节点的本地账本

PBFT 共识算法:

Quorum 组成:

- Client
- Replica

每个共识阶段称为一个 view

- 一个 primary
- 其他 replica 都是 backup

共识过程:

- Request: client 发起共识请求
- prepare: primary 验证请求、打包消息、发给其他节点
- Prepare:其他节点验证消息、投票
- Commit: primary 收到 2f+1 个节点赞成票后, 完成共识, 通知所有节点, 反馈 client

第六部分, 区块链发展与应用

区块链技术问题: 性能问题、安全问题、隐私保护问题、链链互联问题、区块链软件开发生态问题、与其他信息技术集成的问题

共识机制: PoW、PoS、DPoS。

新的共识机制: POS-POW (POW 矿工创建区块, POS 用户确认区块的合法性)、DBFT (由权益来选出记账人, 然后记账人之间通过拜占庭容错算法来达到共识)、BFT-DPOS (以得票数量排序的代理人节点列表中选择固定数量的节点, 其再以 BFT 算法进行共识)、VRF (优势是快速共识、抗攻击能力、极低算力需求)。

隐私保护:

- 零知识证明: : 证明者能够在不向验证者提供任何有用的信息的情况下, 使验证者相信某个论断是正确的
- 同态加密: 对经过同态加密的数据进行处理得到一个输出, 将这一输出进行解密, 其结果与用同一方法处理未加密的原始数据得到的输出结果是一样的

侧链和跨链: 联盟链和公有链 (公证人模式、中继链模式、侧链模式、哈希锁定模式) 多链和多通道 (fabric)

分片技术: 在业务上相互独立, 在绝大多数情况下不需要相互通讯, 每个分片都有自己的共识节点, 使得计算得以真正并行化。

状态通道: 以提高区块链执行效率为目的其基本原理是通过将部分流程转移到链外执行来提高区块链的效率 且不会增加参与者的风险

闪电网络: 不存在交易对手的风险 无需信任对方以及第三方即可实现实时的、海量的交易网络

雷电网络: 类似于闪电网络 参与者在互相转账时不需要通过以太坊主链交易确认而是通过参与者之间创建状态通道在链下完成。即时到账、低转账费用、可大规模扩展、隐私保护

智能合约代码存在的问题:

1. 如何确保合约代码的正确性
2. 合约验证问题
3. 合约的定制问题
4. 一致性问题
5. 执行过程的可控性和可调度性

区块链的应用场景: 数字货币、支付转账、数字票据、证券交易、保险、供应链金融、资产证券化

查漏补缺以及一些例子

分布式账本技术原理:

- 将交易向全网所有节点进行广播
- 众多记账节点对记账权达成共识, 由共识确认的记账节点把记账区块发布给全网
- 所有账本数据完整存储于区块链网络的每个节点
- 所有节点都对账本数据的合法性和完整性进行验证

比特币现金: 作为纽约共识(比特币区块从1M扩容到8M)失败的备份。

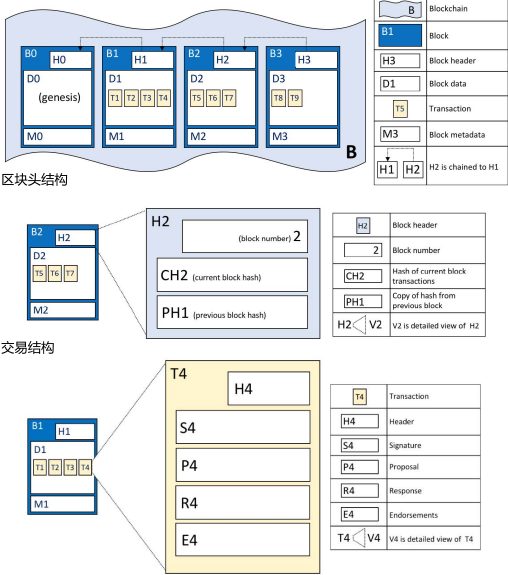
Script 算法: 计算所需时间长, 而且占用的内存也多, 使得并行计算多个摘要异常困难。同时可以抗 ASIC 挖矿。

数字货币钱包:

- SPV钱包包, 只维护与自己相关的区块链数据
- 中心化钱包 (在线钱包), 依赖自己的中心化服务器, 如blockchain.info
- 硬件钱包、冷钱包

The DAO 事件: 在程序员修复递归调用漏洞及其他问题的期间, 一个不知名的黑客开始利用这一途径收集 THE DAO 代币销售中所得的以太坊。6 月 18 日, 黑客成功挖到超过 360 万个以太坊, 并投入到一个 DAO 子组织中, 这个组织和 THE DAO 有着同样的结构。因此, 在块 1,920,000 以太坊实施了 DAO 硬分叉因此创建了两个以太坊: 以太坊和以太坊经典。

Fabric 区块链结构



以太坊交易结构

```
class Transaction {
  byte[] hash; //注意这是对RLP编码之后的交易请求Tx的Hash值。
  byte[] nonce; //实质上是Tx的序号, 用以防止对同一交易请求的重复处理。
  byte[] value; //支付的币值, Wei是以大币的最小单元。
  byte[] receiveAddress; //对方地址, 这地址可以是账户地址或合约地址。
  byte[] gasPrice; //油价
  byte[] gasLimit; //可以付出的最大油量
  byte[] data; //见下页解释
  Integer chainId; //子网/子链ID
  ECDSA_Signature signature; //签名
  byte[] sendAddress; //发出方地址
}
```

以太坊区块数据结构

```
class Block { //Block类是一个数据结构, 内含下面这些结构成分:
  BlockHeader header; //区块头
  List<Transaction> transactionsList //交易记录序列
  List<BlockHeader> unclesList //叔区块的块头序列

  class BlockHeader { //最长可达800字节
    byte[] parentHash; //前母块 (父块) 的块头Hash值
    byte[] unclesHash; //块身中unclesList的Hash值
    byte[] Coinbase; //表示本区块的Coinbase和手续费应该给谁, 其160位地址。
    byte[] stateRoot; //执行完本块所含全部交易后的状态树Hash值。
    byte[] txTrueRoot; //块身中所有交易记录的树根Hash值。
    byte[] receiptTreeRoot; //各个交易收据所构成树根的Hash值。
    byte[] difficulty; //挖矿难度, 用以调整发块周期长度
    long timeStamp; //时间戳
    long number; //本块在区块链中的高度, 即区块链中处于本块之前的区块个数。
    byte[] gasLimit; //本块所含所有交易所提供“汽油”量即手续费的上限总和。
    byte[] gasUsed; //本块所含所有交易实际消耗“汽油”量的总和。
    byte[] extraData; //有关本区块的任意额外数据, 不超过32字节。
    byte[] nonce; //
  }
}
```

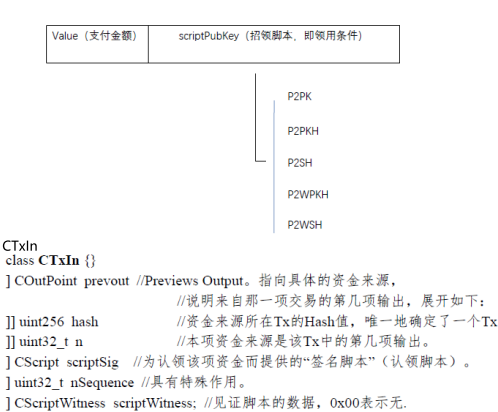
比特币区块头结构

```
class CBlockHeader { //这是一个“类”, 或数据机构, 内含下面这些字段
  int32_t nVersion //所采用Bitcoin协议的版本号
  uint256 hashPrevBlock //上一个块的 (块头) Hash值。
  uint256 hashMerkleRoot //所记载交易记录的TxID,
  //即其Hash值所构成Merkle树的根
  uint32_t nTime //本块的发布时间
  uint32_t nBits //为挖矿过程设置的难度, Hash值中须有的前导零的位数。
  uint32_t nNonce //在挖矿过程中使Hash值达到nBits字段所
  //要求前导零位数的试凑值。
}
```

比特币区块交易结构

```
class CTransaction { //一个交易请求或交易记录
  const std::vector<CTxIn> vin //本Tx的输入UTXO序列, 即资金来源。
  const std::vector<CTxOut> vout //本Tx的输出UTXO序列, 即资金去向。
  const int32_t nVersion //CURRENT_VERSION=2
  const uint32_t nLockTime //锁定时间, 时间未到期之前本交易不入块。
  const uint256 hash //本Tx的Hash值, 只存储在内存中,
  //不作永久存储也不发送。

  CTxOut
  class CTxOut {
  CAmount nValue //本项输出即该UTXO所承载的 (比特币) 价值
  CScript scriptPubKey //招领脚本, 虽然名为scriptPubKey,
  //却未必只是以公钥为条件的脚本
}
```



CTxIn

```
class CTxIn {
  COutPoint prevout //Previus Output. 指向具体的资金来源,
  //说明来自那一项交易的第几项输出, 展开如下:
  //资金来源所在Tx的Hash值, 唯一地确定了一个Tx。
  //本项资金来源是该Tx中的第几项输出。
  CScript scriptSig //为认领该项资金而提供的“签名脚本” (认领脚本)。
  uint32_t nSequence //具有特殊作用。
  CScriptWitness scriptWitness; //见证脚本的数据, 0x00表示无。
}
```

prevout scriptSig Sequence scriptWitness

hash n

本项资金来源所在交易记录的Hash值

本项资金来源是该交易记录中的第几项输出